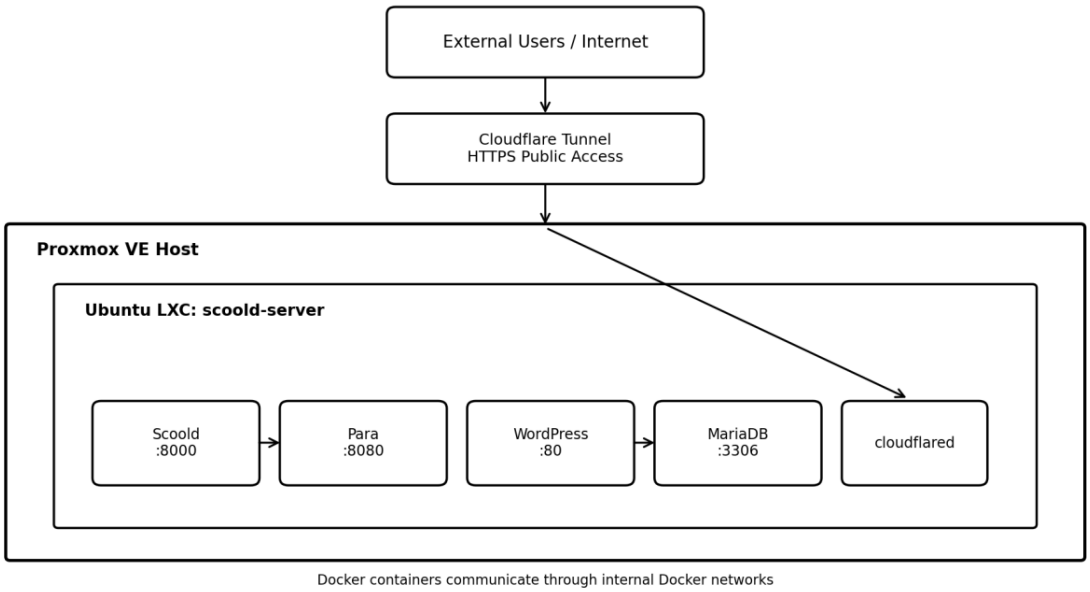
CONTAINERIZED DEPLOYMENT OF SCOOLD Q&A PLATFORM

A Professional Project Report



Student:	Amr Bolas
Instructor:	Dr. Jehad Hamamreh
Course:	Programming Networks
Project Type:	Network Deployment and Systems Integration
Public Scoold URL:	https://app.amrbolas.site
Public Portal URL:	https://portal.amrbolas.site

Abstract

This project presents the design, deployment, publication, networking, and integration of the Scoold Question and Answer platform in a self-hosted environment. A dedicated computer running Proxmox VE was used as the physical server. An Ubuntu LXC container hosted Docker services for Scoold, Para, Cloudflared, WordPress, and MariaDB. Cloudflare Tunnel provided secure HTTPS access without router port forwarding. A Containerlab topology was built on a separate laptop to simulate multiple LANs, routers, an ISP, OSPF dynamic routing, NAT, and an iptables firewall. A custom WordPress plugin was also developed to retrieve live public information from Scoold and display community statistics and recent questions through a shortcode. The final environment demonstrates practical knowledge of virtualization, Linux administration, containerization, routing, firewall configuration, secure public publishing, persistence, and web application integration.

Executive Summary

Infrastructure	Proxmox VE with Ubuntu 24.04 LTS LXC container
Application Services	Scoold, Para, WordPress, MariaDB, and Cloudflared
Container Platform	Docker Engine and Docker Compose
Public Access	Cloudflare Tunnel with HTTPS hostnames
Network Laboratory	Containerlab with FRRouting, OSPF, ISP, firewall, and NAT
Integration	Custom WordPress plugin using public Scoold pages
Final Status	Both public websites and all major tests passed

Table of Contents

1. Introduction

Modern web systems rarely consist of a single application installed directly on one operating system. They normally use virtualization, containers, isolated networks, public gateways, databases, security controls, and automated recovery. The purpose of this project was to apply these ideas in one complete working deployment rather than presenting them only as theoretical concepts.

Scoold is an open-source Question and Answer platform. It provides a community interface for questions, answers, users, tags, profiles, and authentication. Scoold depends on Para as its backend service. The project deployed both services on a self-hosted Proxmox server, made them publicly accessible through Cloudflare Tunnel, tested network connectivity using Containerlab, and integrated the resulting platform with a WordPress portal.

Project idea: build a production-like environment in which the application, the network, the firewall, the public domains, and the WordPress integration are all tested as one system.

2. Project Objectives and Scope

- Create a dedicated and isolated Linux server using Proxmox VE and Ubuntu LXC.
- Deploy Scoold and Para as Docker containers using persistent configuration.
- Publish the services securely through Cloudflare Tunnel without opening router ports.
- Build a multi-network topology in Containerlab with clients, routers, ISP, and firewall nodes.
- Replace initial static routes with OSPF dynamic routing using FRRouting.
- Implement NAT and stateful firewall rules using iptables.
- Deploy WordPress and MariaDB and expose the portal through a second public hostname.
- Develop a custom WordPress plugin that displays live Scoold statistics and recent questions.
- Configure automatic startup and validate recovery after a restart or power interruption.

3. Technologies and Design Decisions

Technology	Purpose	Reason for Selection
Proxmox VE	Virtualization and LXC management	Central management, isolation, and automatic startup
Ubuntu LXC	Main Linux server environment	Lower resource consumption than a full virtual machine
Docker	Application containerization	Isolation, portability, and simplified deployment
Docker Compose	Multi-container orchestration	Repeatable service definitions and one-command startup
Scoold	Q&A web application	Provides the user-facing community platform
Para	Backend for Scoold	Stores users, questions, tags, and application data
Cloudflare Tunnel	Secure public publishing	HTTPS access without port forwarding or static public IP
Containerlab	Virtual network simulation	Creates routers and clients without physical hardware
FRRouting	Dynamic routing	Provides OSPF support on Linux routers
iptables	Firewall and NAT	Stateful filtering and MASQUERADE translation
WordPress	Public project portal	Provides a flexible website and integration point
MariaDB	WordPress database	Compatible alternative after MySQL CPU compatibility failure
PHP / DOMXPath	Custom plugin development	Parses Scoold pages and renders dynamic WordPress content

4. System Architecture

The final solution contains two related environments. The application environment runs on the Proxmox server. The network laboratory runs on the laptop using Containerlab. Separating the environments avoided overloading the resource-constrained server while still allowing complete end-to-end connectivity tests.

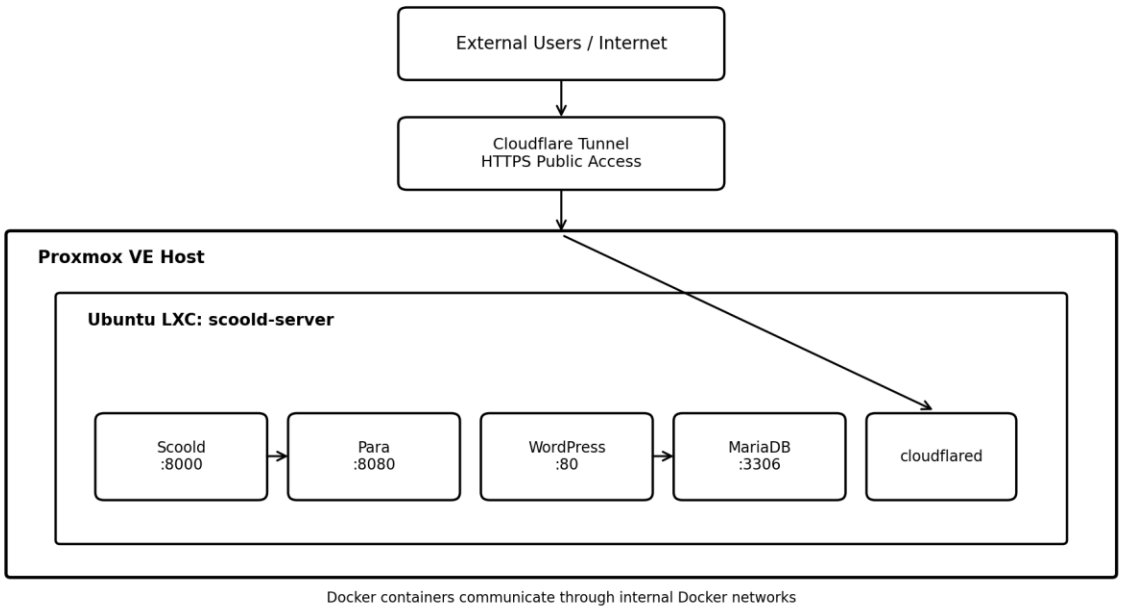


Figure 1. High-level application and infrastructure architecture.

Incoming users access one of two HTTPS hostnames. Cloudflare receives the request and sends it through the encrypted tunnel to the cloudflared container. The request is then forwarded internally to Scoold or WordPress. Scoold communicates with Para, while WordPress communicates with MariaDB.

5. Proxmox and Ubuntu LXC Preparation

5.1 Physical Server and Proxmox VE

A dedicated external computer was converted into the project server by installing Proxmox VE. Proxmox provides a web-based interface for creating, starting, stopping, and monitoring virtual machines and Linux containers. The Proxmox host used the local IP address 192.168.1.24.

5.2 Ubuntu LXC Container

An LXC container named scoold-server with container ID 100 was created. It ran Ubuntu 24.04 LTS and used the local IP address 192.168.1.50. LXC was selected instead of a full virtual machine because it uses fewer CPU and memory resources while still providing an isolated Linux userspace.

5.3 Automatic Startup

The LXC container was configured with Start at boot enabled. This means that after the physical Proxmox machine starts, the application container starts automatically. Docker restart policies then restore the individual services.

6. Docker Deployment of Scoold and Para

6.1 Docker Installation and Service Isolation

Docker Engine and Docker Compose were installed inside the Ubuntu LXC container. Docker was used so that every service could run in an isolated environment with its own image, libraries, settings, storage, and network identity. This design reduces conflicts and makes the deployment reproducible.

6.2 Service Roles

Service	Port	Role
Scoold	8000	User-facing Q&A platform
Para	8080	Backend for users, posts, tags, authentication, and search

6.3 Internal Communication

Scoold reached Para through Docker's internal DNS using the service address http://para:8080. Using a service name instead of a fixed container IP makes the configuration stable even when containers are recreated and receive different internal addresses.

```
docker logs --tail 100 scoold-scoold-1
# Expected: Connected to Para on http://para:8080
```

6.4 Persistent Configuration

Configuration files were created on the LXC host and mounted into the containers. This allowed settings to remain outside the images and made changes possible without rebuilding the containers.

7. Secure Publication with Cloudflare Tunnel

7.1 Why a Tunnel Was Used

Traditional self-hosting normally requires port forwarding on the home router and may require a public static IP address. Cloudflare Tunnel avoids these requirements by creating an outbound encrypted connection from the server to Cloudflare. No inbound router port was opened.

7.2 Public Hostnames

Public Hostname	Internal Service	Purpose
app.amrbolas.site	http://scoold-scoold-1:8000	Public Scoold platform
portal.amrbolas.site	http://wordpress:80	Public WordPress portal

7.3 Correct Application URL

Scoold originally referenced a temporary Cloudflare address. The scoold.host_url setting was changed to https://app.amrbolas.site. This ensured that internal links, redirects, logout actions, cookies, and resources used the final domain.

```
scoold.host_url = "https://app.amrbolas.site"
```

7.4 Verification

```
curl -I https://app.amrbolas.site
curl -I https://portal.amrbolas.site
# Both returned HTTP/2 200
```

8. Containerlab Network Topology

Containerlab was installed on the laptop rather than the Proxmox server to avoid consuming the server's limited memory. The lab simulated a realistic network using Docker-based network nodes.

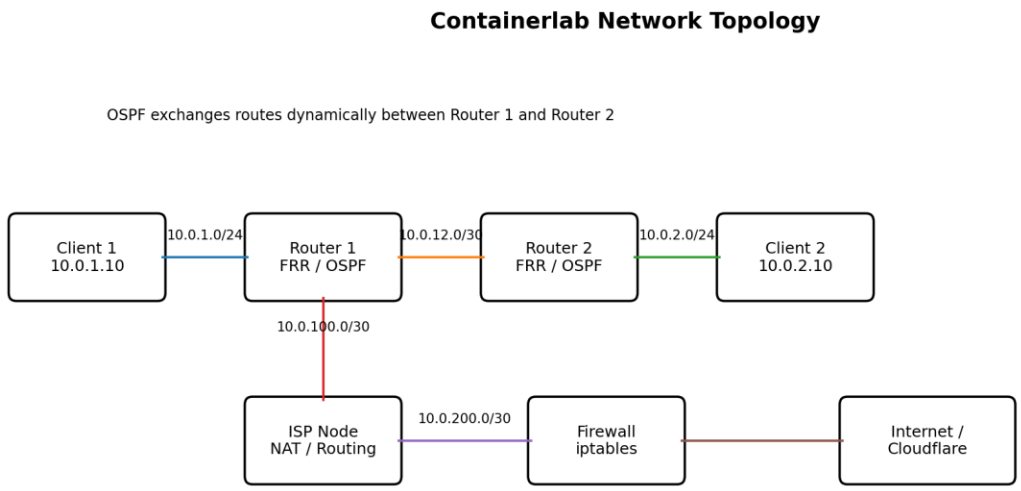


Figure 2. Containerlab topology used for routing and security testing.

8.1 Addressing Plan

Segment	Subnet	Purpose
Client 1 LAN	10.0.1.0/24	Client 1 and Router 1
Router Interconnect	10.0.12.0/30	Point-to-point link between routers
Client 2 LAN	10.0.2.0/24	Router 2 and Client 2
Router 1 to ISP	10.0.100.0/30	External path toward ISP
ISP to Firewall	10.0.200.0/30	Security boundary before Internet

8.2 Initial Static Routing

The first version used static routes to verify basic forwarding between the two LANs. IP forwarding was enabled on both routers. Ping and traceroute confirmed that packets crossed Router 1 and Router 2.

```
sysctl -w net.ipv4.ip_forward=1
```

9. FRRouting, OSPF, NAT, and Firewall

9.1 Migrating from Static Routes to OSPF

Static routes are acceptable for small networks, but they require manual maintenance. Router 1 and Router 2 were replaced with FRRouting containers. The zebra and ospfd daemons were enabled, and the connected networks were advertised through OSPF.

- 10.0.1.0/24 - Client 1 LAN
- 10.0.12.0/30 - Router interconnect
- 10.0.2.0/24 - Client 2 LAN

The router-to-router interface was configured as point-to-point. The OSPF neighbor state reached Full/-, confirming that the routers exchanged link-state databases successfully.

```
vtysh -c "show ip ospf neighbor"
# Neighbor state: Full/-
```

9.2 Default Route Distribution

Router 1 had the path toward the ISP. The command default-information originate advertised this default route into OSPF, allowing Router 2 and Client 2 to reach the Internet dynamically.

9.3 NAT and Internet Access

Private IP addresses cannot be routed directly on the public Internet. A MASQUERADE rule translated traffic from 10.0.0.0/8 to the address of the external interface.

```
iptables -t nat -A POSTROUTING -s 10.0.0.0/8 -o eth0 -j MASQUERADE
```

9.4 Stateful Firewall

A dedicated firewall node was inserted between the ISP and the Internet. The default forwarding policy was set to DROP. Outbound traffic from the internal network was allowed, while inbound return traffic was allowed only for ESTABLISHED or RELATED connections.

```
iptables -P FORWARD DROP
iptables -A FORWARD -s 10.0.0.0/8 -i eth1 -o eth0 -j ACCEPT
iptables -A FORWARD -d 10.0.0.0/8 -i eth0 -o eth1 \
  -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

10. WordPress Deployment and Public Portal

10.1 WordPress and Database Containers

WordPress was deployed with a separate database container. MySQL 8.0 was attempted first, but the image failed because the older CPU did not support x86-64-v2 instructions. MariaDB 10.11 was used as a compatible replacement.

Resolved compatibility issue: mysql:8.0 was replaced with mariadb:10.11.

10.2 Persistence and Credentials

Docker volumes preserved the WordPress files and MariaDB data. Database credentials were stored in a protected .env file with chmod 600 permissions.

10.3 Public URL and Redirect Correction

The portal was published through Cloudflare. WordPress initially redirected to the local :8090 port because the home and siteurl values still contained the installation address. Both database values were changed to https://portal.amrbolas.site.

10.4 Permanent Docker Networking

WordPress needed to share a Docker network with the existing Cloudflare tunnel. The scoold_default network was declared as an external network in compose.yml, so the connection survives container recreation.

```
networks:
  wordpress-network:
  scoold-network:
    external: true
    name: scoold_default
```

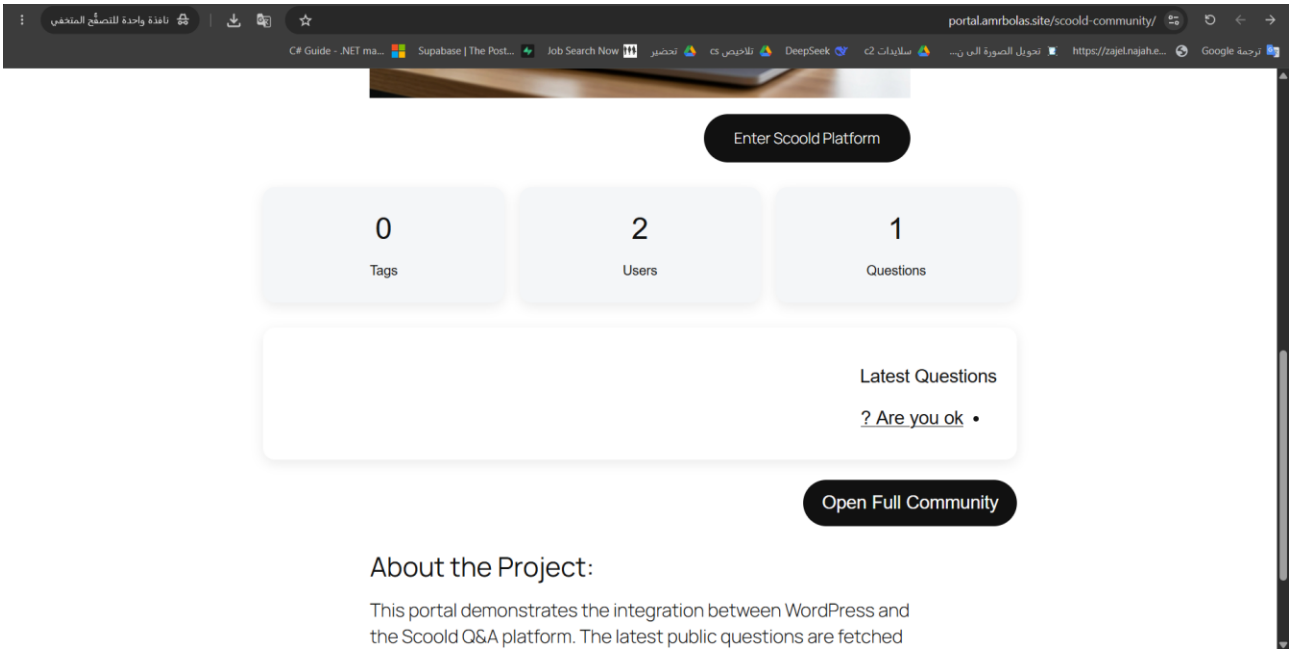


Figure 3. Public WordPress portal displaying Scoold information.

11. Custom Scoold Dashboard Plugin

11.1 Purpose of the Integration

A simple button linking WordPress to Scoold would not demonstrate dynamic integration. A custom plugin was therefore developed to retrieve live information from Scoold and display it directly inside WordPress.

11.2 Data Sources and Processing

The plugin retrieves the public Scoold pages /questions, /people, and /tags. PHP DOMDocument and DOMXPath parse the returned HTML to identify recent question links, user cards, and tag elements.

11.3 Shortcode and Dashboard

The plugin registers the shortcode [scoold_dashboard]. When the shortcode is placed in a WordPress page, it renders cards for the number of questions, users, and tags, followed by a list of recent questions and a button that opens the full Scoold platform.

```
add_shortcode('scoold_dashboard', 'scoold_dashboard_shortcode');
```

11.4 Performance Cache

WordPress Transients cache the downloaded pages for five minutes. This avoids requesting the same Scoold pages for every visitor and reduces load on the low-resource Proxmox server.

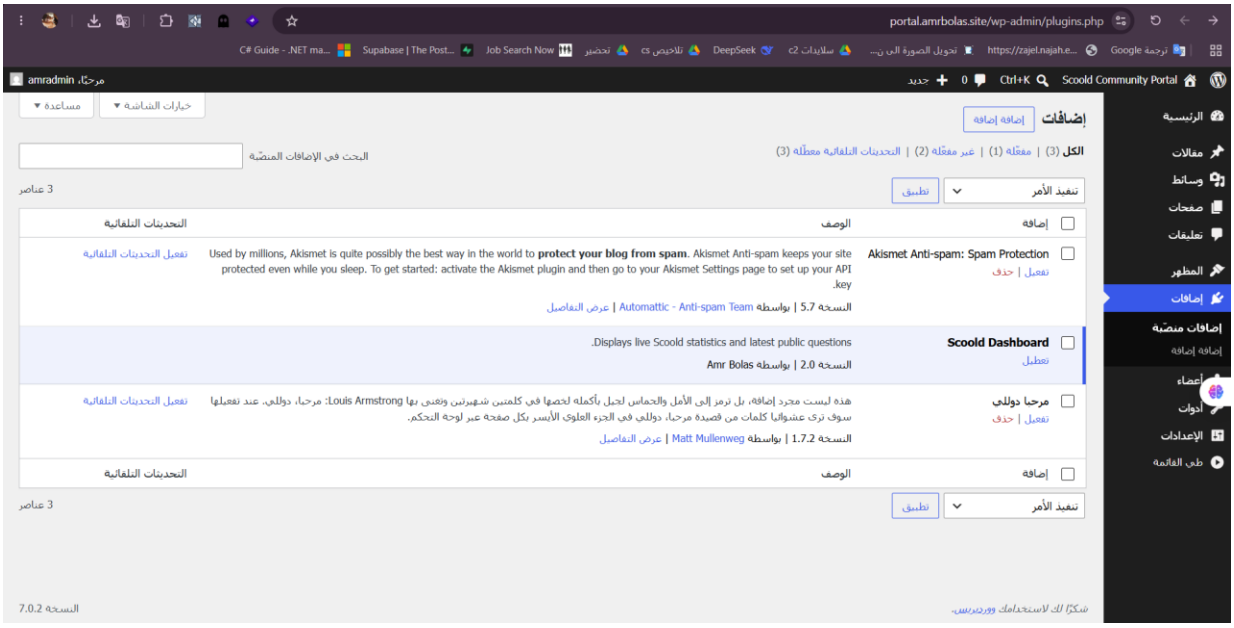


Figure 4. The custom Scoold Dashboard plugin active in WordPress.

12. Testing and Validation

Test	Method	Result
Container state	docker ps and docker compose ps	All required containers running
Scoold to Para	Scoold logs and HTTP requests	Connected successfully
Scoold public access	curl -I https://app.amrbolas.site	HTTP/2 200
WordPress public access	curl -I https://portal.amrbolas.site	HTTP/2 200
OSPF adjacency	show ip ospf neighbor	Full/-
Inter-LAN connectivity	Client 1 ping to Client 2	0% packet loss
Internet access	Client 2 ping 1.1.1.1	Successful
Route path	traceroute to 1.1.1.1	Passed through routers, ISP, and firewall
Firewall/NAT	iptables packet counters	Counters increased
Plugin integration	Create Scoold question and refresh portal	Question/statistics updated
Automatic recovery	Power interruption / restart	Services recovered automatically

13. Challenges and Solutions

Challenge	Cause	Solution
Containerlab topology parsing error	The privileged field was not supported by the installed Containerlab version.	Removed the unsupported field and redeployed the topology.
Incorrect router default route	Docker management routes were selected instead of the OSPF/ISP path.	Deleted the management default route and advertised the correct route through OSPF.
OSPF neighbor stuck at 2-Way	The broadcast network type caused DR/BDR behavior.	Configured the inter-router interfaces as point-to-point.
MySQL CPU incompatibility	mysql:8.0 required x86-64-v2 instructions unavailable on the server.	Replaced MySQL with MariaDB 10.11.
WordPress redirected to :8090	The database still stored the original local installation URL.	Updated the home and siteurl values to the public HTTPS domain.
Cloudflare could not reach WordPress	Cloudflared and WordPress were on different Docker networks.	Connected WordPress to scoold_default and made the network persistent in Compose.
Shortcode displayed as text	The homepage used an older page and cached content.	Selected Scoold Community as the static homepage and refreshed the page/cache.
Write API integration failed	Scoold requires every created question to have an existing author.	Kept the stable read-only integration instead of adding incomplete account synchronization.

14. Security, Reliability, and Persistence

- Cloudflare Tunnel eliminates direct inbound port exposure on the router.
- HTTPS is used for both public websites.
- The firewall uses a default-deny forwarding policy.
- Only established and related return traffic is accepted from the external side.
- The .env database file is restricted with chmod 600.
- Docker volumes preserve WordPress and MariaDB data.
- Docker restart policies recover services after reboots.
- Proxmox starts the LXC container automatically.
- External Docker network declarations survive container recreation.

Together, these controls improve confidentiality, integrity, availability, and operational reliability within the limits of the project environment.

15. Final Results and Conclusion

15.1 Final Deliverables

- A self-hosted Scoold platform available at https://app.amrbolas.site.
- A WordPress portal available at https://portal.amrbolas.site.
- Scoold and Para running as connected Docker services.
- A Containerlab topology with two LANs, FRRouting OSPF, ISP, firewall, NAT, and Internet access.
- A custom WordPress plugin showing live Scoold statistics and recent questions.
- Automatic recovery after restart or power interruption.

15.2 Conclusion

The project successfully combined application deployment and computer networking into one working system. It demonstrated the use of virtualization, Linux containers, Docker Compose, service discovery, secure public access, OSPF, NAT, stateful firewall policies, persistent storage, database administration, and WordPress plugin development. The final result is not only a running web application; it is a complete deployment environment that was designed, tested, secured, and integrated from end to end.

Appendix A. Key Commands and Configuration Checks

Check container status

```
docker ps
```

Check service restart policies

```
docker inspect -f '{{.Name}} -> {{.HostConfig.RestartPolicy.Name}}' $(docker ps -aq)
```

Verify Scoold HTTP response

```
curl -I http://localhost:8000
```

Verify public Scoold

```
curl -I https://app.amrbolas.site
```

Verify public WordPress

```
curl -I https://portal.amrbolas.site
```

Check OSPF neighbor

```
docker exec clab-scoold-network-router1 vtysh -c "show ip ospf neighbor"
```

Check routing table

```
docker exec clab-scoold-network-router2 vtysh -c "show ip route"
```

Test Internet from Client 2

```
docker exec clab-scoold-network-client2 ping -c 4 1.1.1.1
```

View traceroute

```
docker exec clab-scoold-network-client2 traceroute -4 1.1.1.1
```

Check firewall counters

```
docker exec clab-scoold-network-firewall iptables -L FORWARD -n -v
```

Validate WordPress Compose

```
docker compose -f compose.yml config
```

Appendix B. Suggested Future Improvements

- Increase RAM allocated to the LXC container for faster Java and WordPress response times.
- Add automated backups for Para, WordPress, MariaDB, and configuration files.
- Implement monitoring and alerting using Prometheus and Grafana.
- Use a reverse proxy and health checks for additional application resilience.
- Implement full user synchronization between WordPress and Scoold before enabling write operations.
- Add CI/CD automation for configuration and plugin updates.